

CSC633 MACHINE LEARNING



TEXAS A&M UNIVERSITY

Department of Computer
Science & Engineering

CSCE 633: Machine Learning

Optimizing for Neural Networks

Texas A&M University

Outline

- Activation Function
- Backpropagation
- Optimization
- Neural Network Training and Design

Optimization (1)

- Gradient Descent (GD)

Algorithm 1 Batch Gradient Descent at Iteration k

Require: Learning rate ϵ_k

Require: Initial Parameter θ

- 1: **while** stopping criteria not met **do**
 - 2: Compute gradient estimate over N examples:
 - 3: $\hat{\mathbf{g}} \leftarrow +\frac{1}{N} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$
 - 4: Apply Update: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$
 - 5: **end while**
-

- Positive: Gradient estimates are stable
- Negative: Need to compute gradients over the entire training for one update

Optimization (2)

- Stochastic Gradient Descent (SGD)

Algorithm 2 Stochastic Gradient Descent at Iteration k

Require: Learning rate ϵ_k

Require: Initial Parameter θ

- 1: **while** stopping criteria not met **do**
 - 2: Sample example $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ from training set
 - 3: Compute gradient estimate:
 - 4: $\hat{\mathbf{g}} \leftarrow +\nabla_{\theta} L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$
 - 5: Apply Update: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$
 - 6: **end while**
-

- ϵ_k is learning rate at step k
- Sufficient condition to guarantee convergence:

$$\sum_{k=1}^{\infty} \epsilon_k = \infty \text{ and } \sum_{k=1}^{\infty} \epsilon_k^2 < \infty$$

Optimization (3)

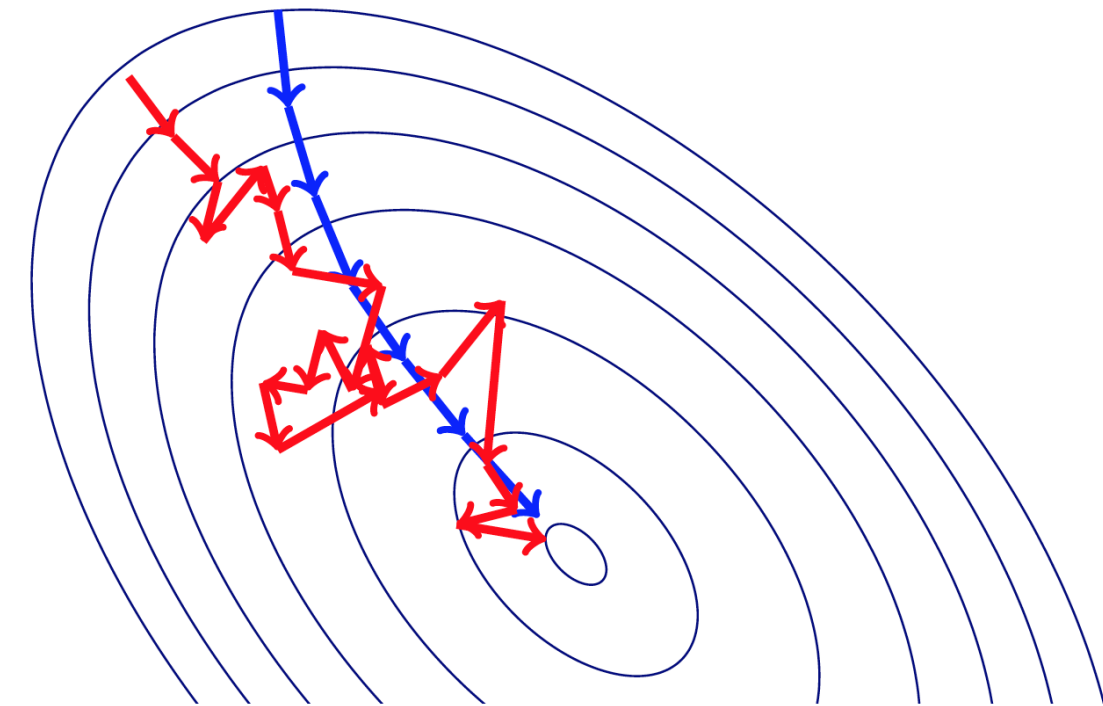
- GD versus SGD

Batch Gradient Descent:

$$\hat{\mathbf{g}} \leftarrow +\frac{1}{N} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$$
$$\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$$

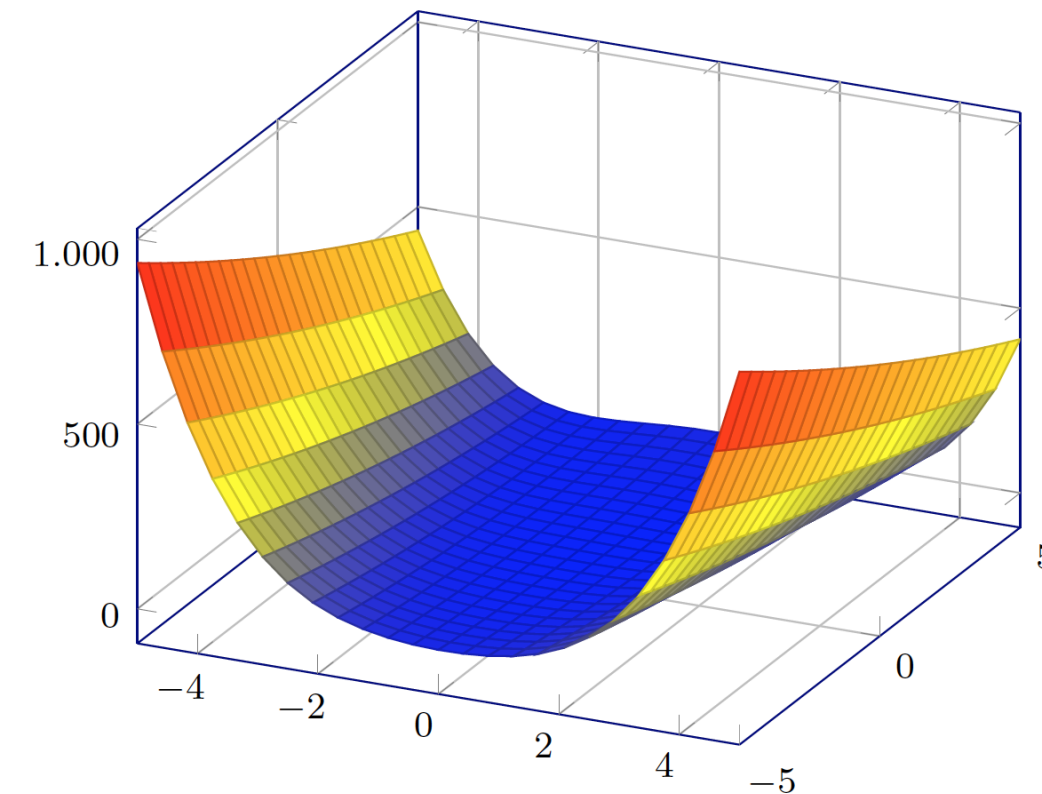
SGD:

$$\hat{\mathbf{g}} \leftarrow +\nabla_{\theta} L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$$
$$\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$$



Optimization (3)

- Momentum
 - The Momentum method is a method to accelerate learning using SGD.
 - In particular SGD suffers in the following scenarios:
 - Error surface has high curvature
 - Small but consistent gradients
 - Noisy gradients



- Gradient Descent would move quickly down the walls, but very slowly through the valley floor

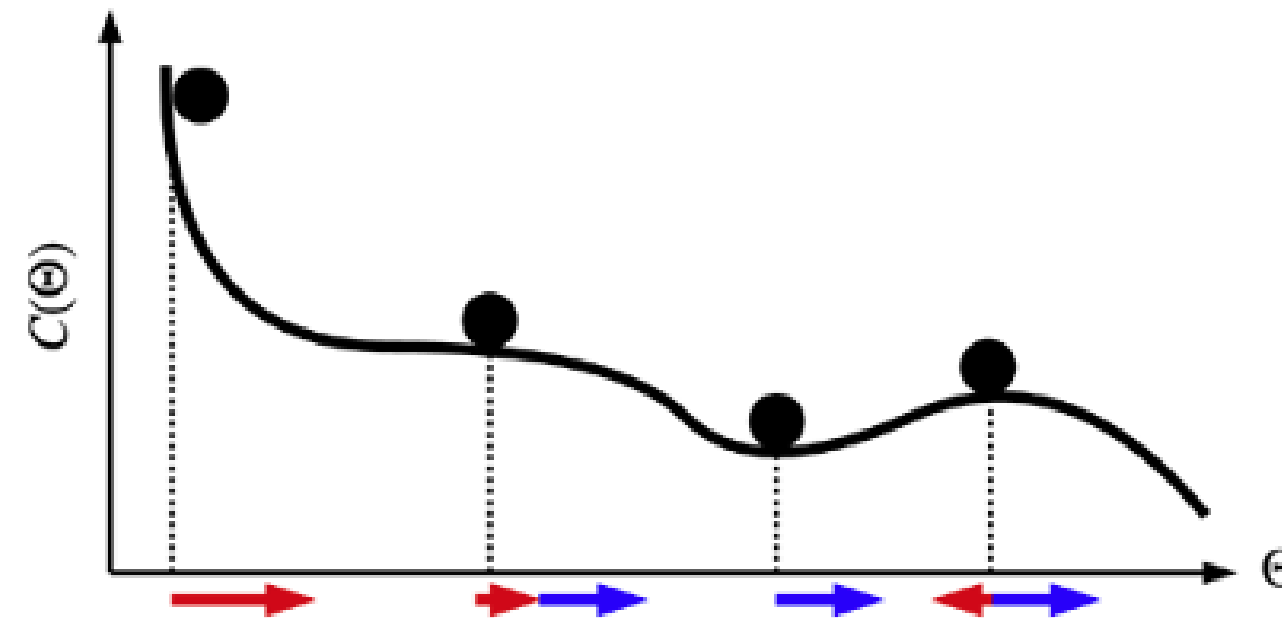
Optimization (4)

Update rule in SGD:

$$\Theta^{(t+1)} \leftarrow \Theta^{(t)} - \eta \mathbf{g}^{(t)}$$

where $\mathbf{g}^{(t)} = \nabla_{\Theta} C(\Theta^{(t)})$

- Gets stuck in local minima or saddle points

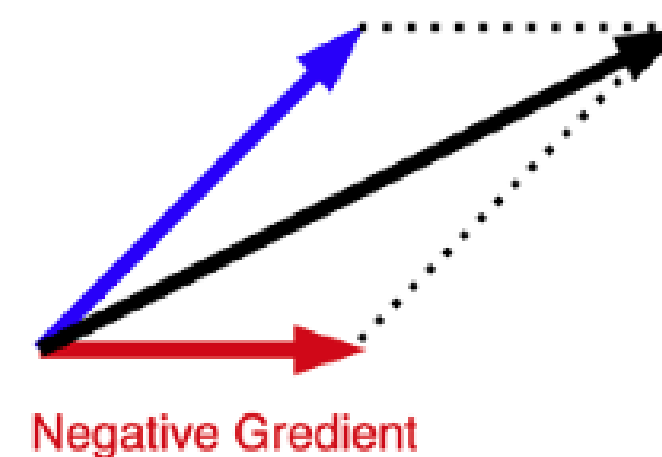


Momentum: make the same movement $\mathbf{v}^{(t)}$ in the last iteration, corrected by negative gradient:

$$\mathbf{v}^{(t+1)} \leftarrow \lambda \mathbf{v}^{(t)} - (1 - \lambda) \mathbf{g}^{(t)}$$

$$\Theta^{(t+1)} \leftarrow \Theta^{(t)} + \eta \mathbf{v}^{(t+1)}$$

$\mathbf{v}^{(t)}$ is a moving average of $-\mathbf{g}^{(t)}$



Optimization (5)

- Popular Solver Examples: AdGrad, RMSProp, Adam

$$\text{SGD: } \theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$$

$$\text{Momentum: } \mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \hat{\mathbf{g}} \text{ then } \theta \leftarrow \theta + \mathbf{v}$$

$$\text{Nesterov: } \mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\theta} \left(L(f(\mathbf{x}^{(i)}; \theta + \alpha \mathbf{v}), \mathbf{y}^{(i)}) \right) \text{ then } \theta \leftarrow \theta + \mathbf{v}$$

$$\text{AdaGrad: } \mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g} \text{ then } \Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g} \text{ then } \theta \leftarrow \theta + \Delta\theta$$

$$\text{RMSProp: } \mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g} \text{ then } \Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g} \text{ then } \theta \leftarrow \theta + \Delta\theta$$

$$\text{Adam: } \hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}, \hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t} \text{ then } \Delta\theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}} \text{ then } \theta \leftarrow \theta + \Delta\theta$$

Optimization (6)

- AdaGrad
 - **Idea:** Downscale a model parameter by square-root of sum of squares of all its historical values
 - Parameters that have large partial derivative of the loss -> learning rates for them are rapidly declined
 - Some interesting theoretical properties

Algorithm 4 AdaGrad

Require: Global Learning rate ϵ , Initial Parameter θ , δ

Initialize $\mathbf{r} = 0$

- 1: **while** stopping criteria not met **do**
 - 2: Sample example $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ from training set
 - 3: Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\nabla_{\theta} L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$
 - 4: Accumulate: $\mathbf{r} \leftarrow \mathbf{r} + \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$
 - 5: Compute update: $\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}$
 - 6: Apply Update: $\theta \leftarrow \theta + \Delta\theta$
 - 7: **end while**
-

Optimization (7)

- RMSProp
 - AdaGrad might shrink the learning rate too aggressively, we can adapt it to perform better by accumulating an exponentially decaying average of the gradient

Algorithm 5 RMSProp

Require: Global Learning rate ϵ , decay parameter ρ , δ

Initialize $\mathbf{r} = 0$

- 1: **while** stopping criteria not met **do**
 - 2: Sample example $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ from training set
 - 3: Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\nabla_{\theta} L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$
 - 4: Accumulate: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$
 - 5: Compute update: $\Delta \theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}$
 - 6: Apply Update: $\theta \leftarrow \theta + \Delta \theta$
 - 7: **end while**
-

Optimization (8)

- Adam
 - Adam is like RMSProp with Momentum but with bias correction terms for the first and second moments

Algorithm 7 RMSProp with Nesterov

Require: ϵ (set to 0.0001), decay rates ρ_1 (set to 0.9), ρ_2 (set to 0.9), θ , δ

Initialize moments variables $\mathbf{s} = 0$ and $\mathbf{r} = 0$, time step $t = 0$

- 1: **while** stopping criteria not met **do**
 - 2: Sample example $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ from training set
 - 3: Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\nabla_{\theta} L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$
 - 4: $t \leftarrow t + 1$
 - 5: Update: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \hat{\mathbf{g}}$
 - 6: Update: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$
 - 7: Correct Biases: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}, \hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$
 - 8: Compute Update: $\Delta\theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$
 - 9: Apply Update: $\theta \leftarrow \theta + \Delta\theta$
 - 10: **end while**
-